LAPORAN PRAKTIKUM
PRAKTIKUM PENAMBANGAN DATA
PERTEMUAN 7
IMBALANCED DATASET DAN FEATURE SELECTION



Disusun oleh:

Nama	: Hafidz Rizqullah Prasetya
NIM	: 24/535493/SV/24243
Kelas	: PL5A1
Dosen Pengampu	: Dr. Imam Fahrurrozi, S.T., M.Cs.

PROGRAM STUDI D-IV
TEKNOLOGI REKAYASA PERANGKAT LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA
2026

Daftar Isi

Daftar Isi **2**
Tujuan Praktikum **3**
Dasar Teori **4**
2.1 Imbalanced Class 4
2.2 Confusion Matrix 5
2.3 Feature Selection 5
2.4 Filter Method 5
2.5 Wrapper Method 6
2.6 Embedded Method 6
2.7 SMOTE 7
2.8 ADASYN 7
Alat dan Bahan **8**
Hasil dan Pembahasan **9**
4.1 Percobaan Latihan Praktikum 9
4.1.1 Persiapan dan Impor Pustaka 9
4.1.2 Memuat Stroke Prediction Dataset 10
4.1.3 Pemeriksaan Nilai Kosong 11
4.1.4 Distribusi Kelas Target 11
4.1.5 Data Cleaning 12
4.1.6 Pembagian Data dan Random Under-Sampling 12
4.1.7 Random Over-Sampling 14
4.1.8 SMOTE 15
4.1.9 ADASYN 16
4.1.10 Seleksi Fitur dengan Filter ANOVA 17
4.1.11 Seleksi Fitur dengan Wrapper RFE 19
4.1.12 Seleksi Fitur dengan Embedded ExtraTree 20
4.1.13 Komparasi Metode Seleksi Fitur dan Pemeriksaan Overfitting . . . 20
4.2 Tugas dan Analisis Praktikum 23
4.2.1 Tugas 1: Eksplorasi Dataset Stroke 23
4.2.2 Tugas 2: Komparasi Balancing pada Dataset Kedua 24
Kesimpulan **28**
Daftar Pustaka **29**

Tujuan Praktikum

Berdasarkan Modul Pertemuan 7 Praktikum Penambangan Data, tujuan praktikum ini adalah:

1. Mahasiswa memahami konsep *imbalanced dataset* serta permasalahan yang muncul akibat ketidakseimbangan kelas dalam model *machine learning*.
2. Mahasiswa mampu menerapkan teknik *balancing data* seperti *Random Under-Sampling*, *Random Over-Sampling*, SMOTE, dan ADASYN untuk menyeimbangkan distribusi kelas pada dataset.
3. Mahasiswa memahami konsep dan metode *feature selection* (*filter*, *wrapper*, *embedded*) untuk memilih fitur yang paling relevan.

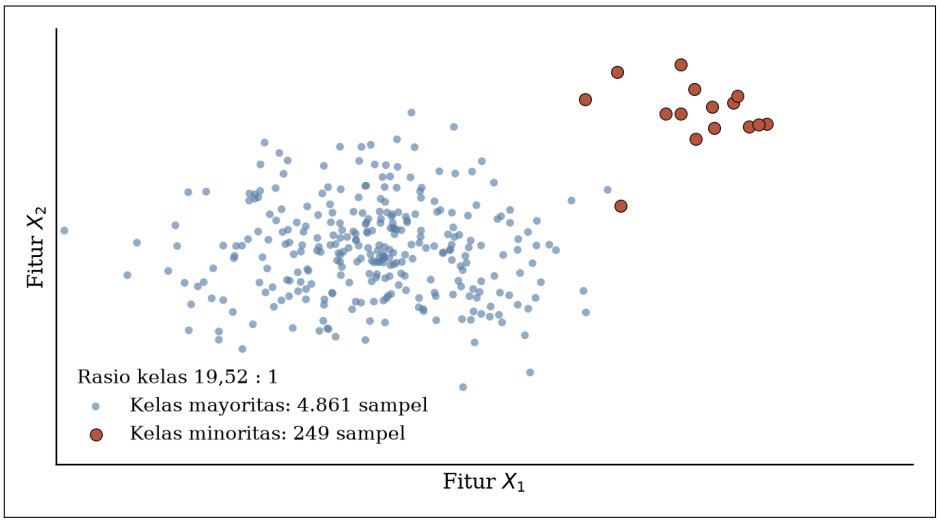
Dasar Teori

2.1 Imbalanced Class

Imbalanced class adalah kondisi ketika satu kelas pada suatu himpunan data memiliki jumlah sampel jauh lebih besar dibanding kelas lainnya. Kelas dengan sampel banyak disebut kelas mayoritas, sedangkan kelas dengan sampel sedikit disebut kelas minoritas. Ketidakseimbangan ini membuat algoritma klasifikasi standar cenderung memprediksi kelas mayoritas dan melewatkan kelas minoritas [1].

Yang lebih menentukan adalah kelas mana yang terabaikan. Pada kasus deteksi stroke, penipuan kartu kredit, atau kegagalan mesin, kelas minoritas justru menjadi kelas yang paling perlu dikenali. Model yang selalu menebak "tidak berisiko" bisa mencapai akurasi 95% pada dataset stroke tanpa benar-benar belajar apa pun. Nilai *recall* kelas minoritas pada kondisi seperti ini mendekati nol [1].

Penanganan ketidakseimbangan kelas bisa ditempuh lewat tiga jalur: memanipulasi data pada level sampel (*resampling*), memodifikasi algoritma lewat pembobotan biaya (*cost-sensitive learning*), atau menggabungkan keduanya. Gambar 1 memperlihatkan bentuk khas sebaran dua kelas yang tidak seimbang pada ruang dua fitur.



Gambar 1: Ilustrasi ketidakseimbangan kelas pada ruang dua fitur. Kelas mayoritas memenuhi sebagian besar ruang, sedangkan kelas minoritas hanya menempati sebagian kecil area.

2.2 Confusion Matrix

Confusion matrix adalah tabel kontingensi yang membandingkan label aktual dengan hasil prediksi model [2]. Tabel 1 memuat empat komponen dasarnya.

Tabel 1: Susunan *confusion matrix* untuk klasifikasi biner.

	Prediksi positif	**Prediksi negatif**
Aktual positif	*True Positive* (TP)	*False Negative* (FN)
Aktual negatif	*False Positive* (FP)	*True Negative* (TN)

Keempat nilai tersebut menjadi dasar perhitungan metrik evaluasi:

$$\textit{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{1}$$

$$\textit{Precision} = \frac{TP}{TP + FP} \tag{2}$$

$$\textit{Recall} = \frac{TP}{TP + FN} \tag{3}$$

$$\textit{F1} = \frac{2 \cdot \textit{Precision} \cdot \textit{Recall}}{\textit{Precision} + \textit{Recall}} \tag{4}$$

Pada data yang tidak seimbang, akurasi menjadi metrik yang menyesatkan. Prediksi yang selalu memilih kelas mayoritas tetap menghasilkan akurasi tinggi karena TN mendominasi perhitungan. *Recall* dan *F1-score* lebih menggambarkan kemampuan model mengenali kelas minoritas, sebab keduanya hanya menghitung TP terhadap jumlah sampel positif yang sebenarnya ada [3].

2.3 Feature Selection

Feature selection adalah proses memilih subset fitur paling relevan dari seluruh fitur yang tersedia sebelum model dibangun [4]. Ketika dimensi fitur terlalu besar, model rentan mengalami *overfitting* dan waktu pelatihan naik tajam. Seleksi fitur memangkas dimensi ruang data dengan membuang fitur yang redundan atau tidak berhubungan dengan target [5].

Manfaatnya meliputi pengurangan waktu komputasi, peningkatan akurasi, dan pemahaman yang lebih baik atas model yang terbentuk. Tiga pendekatan yang umum dipakai dirangkum pada Tabel 2.

2.4 Filter Method

Filter method mengevaluasi relevansi fitur dari sifat statistiknya terhadap variabel target, tanpa melibatkan model *machine learning* [5]. Ukuran yang lazim dipakai adalah korelasi Pearson atau Spearman, uji Chi-Square untuk fitur kategorikal, dan uji ANOVA untuk fitur numerik.

Tabel 2: Perbandingan tiga pendekatan seleksi fitur.

Pendekatan	**Dasar penilaian**	**Biaya komputasi**	**Kekuatan**
Filter	Statistik fitur terhadap target, tanpa melatih model	Ringan, satu kali hitung	Cepat dan mudah ditafsirkan
Wrapper	Performa model pada setiap kombinasi fitur	Berat, model dilatih berulang	Menangkap interaksi antar-fitur
Embedded	Seleksi berjalan di dalam proses pelatihan	Sedang	Seimbang antara akurasi dan waktu

Uji ANOVA (*Analysis of Variance*) membandingkan rata-rata suatu fitur antar-kelompok kelas. Nilai F yang besar berarti rata-rata fitur tersebut berbeda nyata antar-kelas, sehingga fitur dinilai informatif untuk memisahkan kelas. Karena prosesnya hanya menghitung satu statistik per fitur, *filter method* selesai dalam hitungan detik bahkan pada dataset berukuran besar.

2.5 Wrapper Method

Wrapper method memakai algoritma *machine learning* secara langsung sebagai fungsi penilai [6]. Metode ini mencoba berbagai kombinasi fitur, melatih model pada setiap kombinasi, lalu memilih kombinasi dengan performa terbaik. Tiga varian yang umum adalah *Forward Selection* (menambah fitur satu per satu), *Backward Elimination* (membuang fitur satu per satu), dan *Recursive Feature Elimination* (RFE).

RFE bekerja dengan melatih model, mengurutkan fitur menurut besarnya bobot, membuang fitur dengan bobot terkecil, lalu mengulang proses tersebut sampai jumlah fitur yang tersisa sesuai target [7]. Pendekatan ini lebih akurat dibanding *filter* karena ikut memperhitungkan interaksi antar-fitur, tetapi biaya komputasinya jauh lebih besar sebab model dilatih berulang kali.

2.6 Embedded Method

Embedded method menjalankan seleksi fitur bersamaan dengan proses pelatihan model. Contoh paling dikenal adalah regresi LASSO dengan regularisasi L_1 yang memaksa koefisien fitur tidak signifikan menjadi tepat nol [8], serta model berbasis pohon seperti *Random Forest* dan *Extra Trees* yang menghitung *feature importance* dari penurunan *impurity* pada setiap pemisahan node [9].

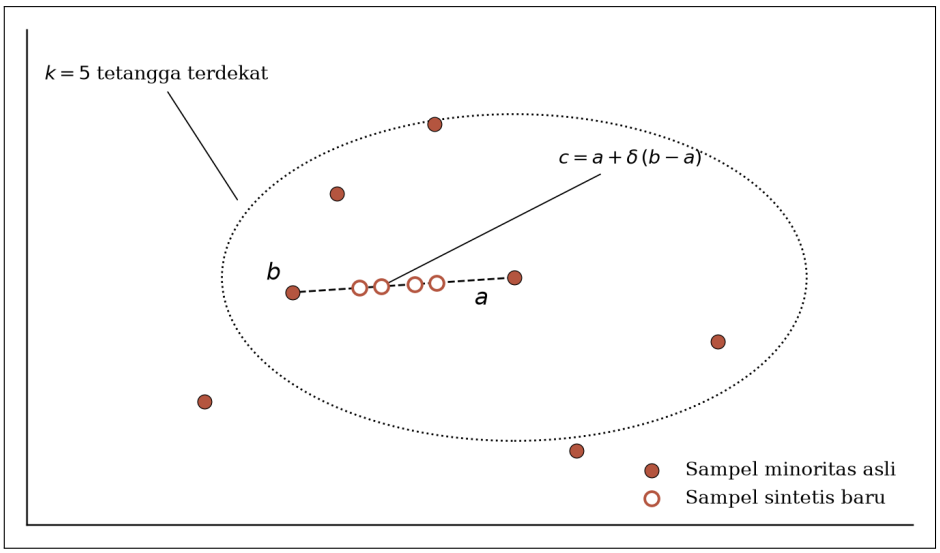
Pendekatan ini menggabungkan kelebihan dua metode sebelumnya: interaksi antar-fitur tetap diperhitungkan seperti *wrapper*, sementara biaya komputasinya mendekati *filter* karena seleksi menumpang pada satu kali pelatihan model.

2.7 SMOTE

Synthetic Minority Over-sampling Technique (SMOTE) membangkitkan sampel minoritas baru, bukan menduplikasi sampel yang sudah ada [10]. Untuk setiap sampel minoritas a, SMOTE mencari k tetangga terdekat dari kelas yang sama (umumnya $k = 5$), memilih satu tetangga b secara acak, lalu membuat sampel sintetis c pada titik acak di sepanjang garis yang menghubungkan a dan b:

$$c = a + \delta\,(b - a), \qquad \delta \sim U(0, 1) \tag{5}$$

Karena titik sintetis lahir di antara sampel asli, kelas minoritas menjadi lebih rapat tanpa kehilangan informasi seperti pada *under-sampling* [12]. Gambar 2 menggambarkan proses tersebut.



Gambar 2: Cara kerja SMOTE. Sampel a dipilih, lima tetangga terdekatnya ditandai, satu tetangga b dipilih acak, lalu sampel sintetis c dibangkitkan pada garis penghubung.

2.8 ADASYN

Adaptive Synthetic Sampling Approach (ADASYN) adalah pengembangan SMOTE yang memperhitungkan tingkat kesulitan setiap sampel minoritas [11]. ADASYN menghitung rasio $r_i = \Delta_i / K$ untuk setiap sampel minoritas, dengan Δ_i sebagai jumlah tetangga terdekat yang berasal dari kelas mayoritas. Rasio tersebut dinormalisasi menjadi $\hat{r}_i = r_i / \sum_i r_i$, lalu jumlah sampel sintetis yang dibangkitkan untuk sampel ke-i adalah $g_i = \hat{r}_i \cdot G$, dengan G sebagai total sampel sintetis yang dibutuhkan.

Akibatnya, sampel minoritas yang berada dekat batas keputusan menerima porsi sintetis lebih besar, sedangkan sampel di area yang mudah dipisahkan menerima porsi lebih kecil. Perbedaan ini yang membedakan ADASYN dari SMOTE yang memperlakukan semua sampel minoritas secara setara [13].

Alat dan Bahan

Sesuai Modul Pertemuan 7 Praktikum Penambangan Data, alat dan bahan yang digunakan dalam praktikum ini meliputi:

A. Perangkat Keras

1. Laptop atau komputer personal (PC) dengan kapasitas memori minimal 4 GB RAM (direkomendasikan 8 GB atau lebih).
2. Koneksi internet yang stabil untuk mengakses lingkungan komputasi daring dan mengunduh berkas dataset.

B. Perangkat Lunak

1. Sistem operasi modern (Windows, macOS, atau Linux).
2. Peramban web (Google Chrome, Mozilla Firefox, atau Microsoft Edge).
3. Lingkungan pengembangan interaktif Python: Google Colaboratory dan Jupyter Notebook lokal.
4. Bahasa pemrograman Python versi 3.8 ke atas [22].
5. Pustaka pemrograman Python:
    - `pandas` dan `numpy` untuk manipulasi data tabular dan operasi matriks numerik [23, 24].
    - `matplotlib.pyplot` dan `seaborn` untuk pembuatan grafik dua dimensi [25, 26].
    - `scikit-learn` untuk pengkodean label, pemodelan pohon keputusan, seleksi fitur (`f_classif`, `RFE`, `ExtraTreesClassifier`), dan metrik evaluasi [20, 15].
    - `imbalanced-learn` untuk penyeimbang kelas `RandomUnderSampler`, `RandomOverSampler`, `SMOTE`, dan `ADASYN` [21, 14].

C. Dataset

1. *Stroke Prediction Dataset* dengan 5.110 baris dan 12 kolom, dipakai pada percobaan penyeimbangan kelas [17].
2. *Heart Attack Analysis and Prediction Dataset* dengan 303 baris dan 14 kolom, dipakai pada percobaan seleksi fitur [18].
3. *Wine Quality (red)* dengan 1.599 baris dan 12 kolom, dipakai pada Tugas 2 [19, 16].

Hasil dan Pembahasan

4.1 Percobaan Latihan Praktikum

Percobaan latihan mengikuti sembilan langkah pada Modul Pertemuan 7, lalu dilanjutkan dengan praktik seleksi fitur dari materi kuliah pertemuan yang sama. Percobaan pertama memakai *Stroke Prediction Dataset* untuk penyeimbangan kelas, sedangkan percobaan seleksi fitur memakai *Heart Attack Analysis and Prediction Dataset*. Seluruh eksperimen dijalankan pada Jupyter Notebook lokal dengan Python 3 dan pustaka `imbalanced-learn` versi 0.14.

Notebook yang sama bisa dibuka dan dijalankan ulang secara interaktif melalui Google Colaboratory pada tautan berikut:

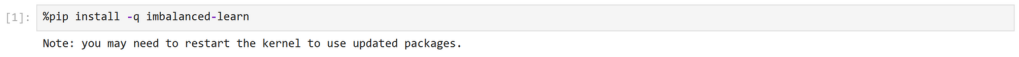
https://colab.research.google.com/github/hafidzrizqullahprasetya/PPD/blob/main/pertemuan-7/praktikum-7-imbalanced-feature-selection.ipynb [27]

4.1.1 Persiapan dan Impor Pustaka

Tahap awal menginstal pustaka `imbalanced-learn` yang menyediakan empat teknik penyeimbangan kelas, lalu mengimpor pustaka manipulasi data, visualisasi, pemodelan, seleksi fitur, dan metrik evaluasi.

```
%pip install -q imbalanced-learn
```

Listing 1: Instalasi pustaka penyeimbang kelas.



Gambar 3: Tangkapan layar instalasi pustaka `imbalanced-learn` pada Jupyter Notebook.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

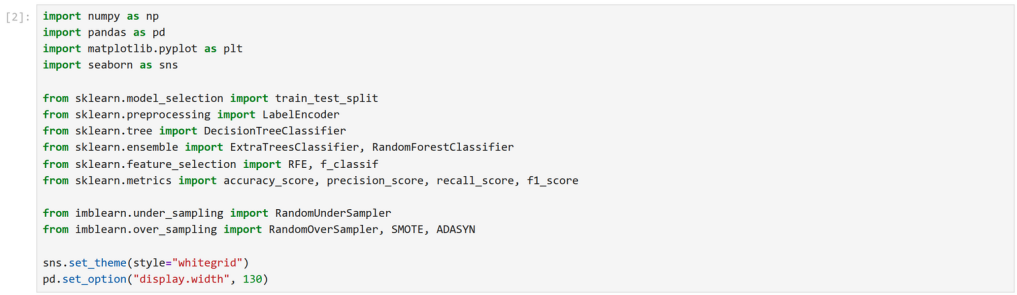
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
```

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import ExtraTreesClassifier, RandomForestClassifier
from sklearn.feature_selection import RFE, f_classif
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

from imblearn.under_sampling import RandomUnderSampler
from imblearn.over_sampling import RandomOverSampler, SMOTE, ADASYN

sns.set_theme(style="whitegrid")
pd.set_option("display.width", 130)
```

Listing 2: Impor pustaka praktikum.



Gambar 4: Tangkapan layar impor pustaka utama pada Jupyter Notebook.

4.1.2 Memuat Stroke Prediction Dataset

Dataset dibaca langsung dari repositori daring sehingga notebook dapat dijalankan ulang tanpa unggah berkas manual. Hasil pembacaan menunjukkan 5.110 baris dan 12 kolom.

```
URL_STROKE = ("https://raw.githubusercontent.com/ganjar87/"
              "data_science_practice/main/healthcare-dataset-stroke-data.csv")
stroke = pd.read_csv(URL_STROKE)
print("Dimensi dataset:", stroke.shape)
stroke.head()
```

Listing 3: Membaca Stroke Prediction Dataset.

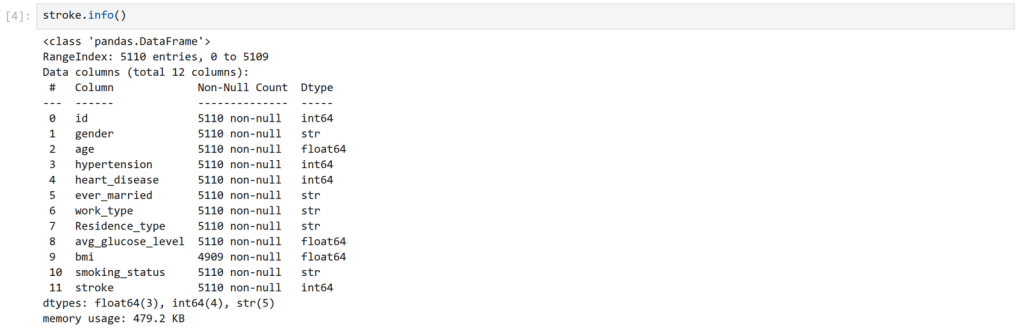


	id	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status	stroke
0	9046	Male	67.0	0	1	Yes	Private	Urban	228.69	36.6	formerly smoked	1
1	51676	Female	61.0	0	0	Yes	Self-employed	Rural	202.21	NaN	never smoked	1
2	31112	Male	80.0	0	1	Yes	Private	Rural	105.92	32.5	never smoked	1
3	60182	Female	49.0	0	0	Yes	Private	Urban	171.23	34.4	smokes	1
4	1665	Female	79.0	1	0	Yes	Self-employed	Rural	174.12	24.0	never smoked	1

Gambar 5: Tangkapan layar pembacaan data dan lima baris pertama dataset stroke.

Pemeriksaan struktur data memakai `stroke.info()` memperlihatkan komposisi tipe data: empat kolom bilangan bulat, tiga kolom bilangan pecahan, dan lima kolom kategorikal

bertipe teks. Kolom `bmi` tercatat hanya memiliki 4.909 nilai terisi dari 5.110 baris, tanda ada nilai kosong yang harus ditangani.



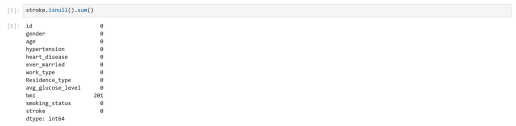
Gambar 6: Tangkapan layar struktur tipe data dan jumlah nilai terisi setiap kolom.

4.1.3 Pemeriksaan Nilai Kosong

Pemeriksaan nilai kosong memastikan kolom mana yang perlu diimputasi sebelum penyeimbangan dijalankan.

```
stroke.isnull().sum()
```

Listing 4: Menghitung nilai kosong setiap kolom.



Gambar 7: Tangkapan layar jumlah nilai kosong per kolom sebelum pembersihan.

Hasilnya hanya kolom `bmi` yang memiliki nilai kosong, sebanyak 201 baris. Sebelas kolom lain terisi penuh.

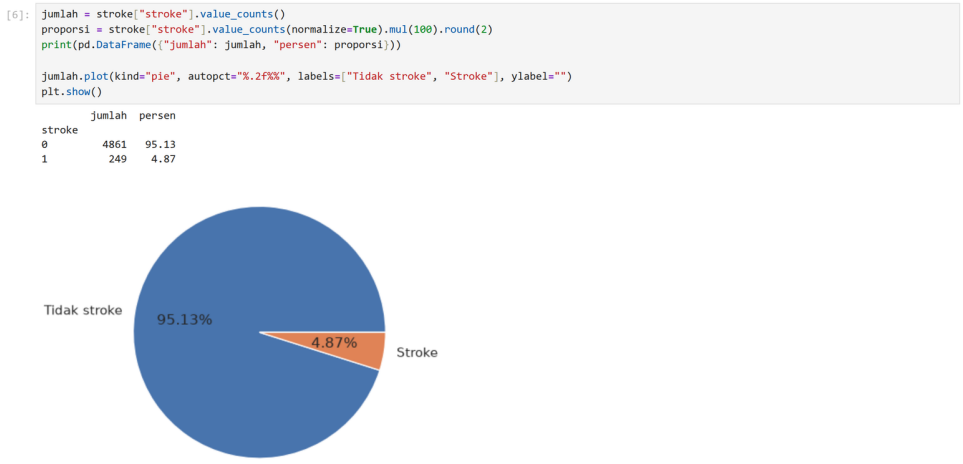
4.1.4 Distribusi Kelas Target

Proporsi kelas target dihitung dan digambarkan sebagai diagram lingkaran untuk melihat besarnya ketidakseimbangan.

```
jumlah = stroke["stroke"].value_counts()
proporsi = stroke["stroke"].value_counts(normalize=True).mul(100).round(2)
print(pd.DataFrame({"jumlah": jumlah, "persen": proporsi}))

jumlah.plot(kind="pie", autopct="%.2f%%", labels=["Tidak stroke", "Stroke"], ylabel="")
plt.show()
```

Listing 5: Menghitung proporsi kelas dan menggambar diagram lingkaran.



Gambar 8: Tangkapan layar jumlah, persentase, dan diagram lingkaran distribusi kelas `stroke`.

Dari 5.110 pasien, 4.861 di antaranya (95,13%) tidak mengalami stroke dan hanya 249 pasien (4,87%) yang mengalami stroke. Rasio kelas mayoritas terhadap kelas minoritas mencapai 19,52 banding 1. Angka ini menjelaskan mengapa penyeimbangan kelas diperlukan: model yang selalu menebak "tidak stroke" sudah mendapat akurasi 95,13% tanpa mempelajari pola apa pun, sementara 249 pasien berisiko tinggi akan terlewat seluruhnya.

4.1.5 Data Cleaning

Nilai kosong pada kolom `bmi` diisi dengan rata-rata kolom tersebut. Pemilihan rata-rata didasarkan pada sifat BMI sebagai variabel kontinu dengan sebaran yang cukup simetris, sehingga nilai tengahnya mewakili populasi secara wajar. Setelah imputasi, pemeriksaan ulang memastikan tidak ada nilai kosong tersisa.

```
stroke["bmi"] = stroke["bmi"].fillna(stroke["bmi"].mean())
stroke.isnull().sum()
```

Listing 6: Imputasi nilai kosong `bmi` dengan rata-rata.



Gambar 9: Tangkapan layar verifikasi data setelah imputasi. Seluruh kolom terisi penuh.

4.1.6 Pembagian Data dan Random Under-Sampling

Fitur dipisahkan dari label target, fitur kategorikal dikodekan menjadi bilangan bulat dengan `LabelEncoder`, lalu data dibagi menjadi set pelatihan dan pengujian dengan rasio

75:25. Penyeimbangan hanya diterapkan pada set pelatihan. Set pengujian dibiarkan apa adanya agar evaluasi tetap mencerminkan sebaran asli di lapangan.

```
def siapkan_data(frame):
    fitur = frame.drop(columns=["id", "stroke"])
    target = frame["stroke"]
    encoder = LabelEncoder()
    for kolom in fitur.select_dtypes(exclude="number").columns:
        fitur[kolom] = encoder.fit_transform(fitur[kolom])
    return train_test_split(fitur.astype(float), target, test_size=0.25, random_state=42)


X_train, X_test, y_train, y_test = siapkan_data(stroke)
print("Latih:", X_train.shape, "| Uji:", X_test.shape)
print(pd.Series(y_train).value_counts())
```

Listing 7: Fungsi penyiapan data dan pembagian data latih-uji.



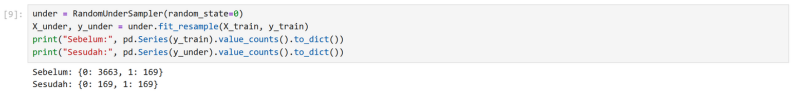
Gambar 10: Tangkapan layar dimensi set pelatihan, set pengujian, dan distribusi kelas pada set pelatihan.

Set pelatihan berisi 3.832 baris dengan komposisi 3.663 sampel kelas 0 dan 169 sampel kelas 1. Set pengujian berisi 1.278 baris.

Random Under-Sampling memangkas sampel kelas mayoritas secara acak sampai jumlahnya setara dengan kelas minoritas.

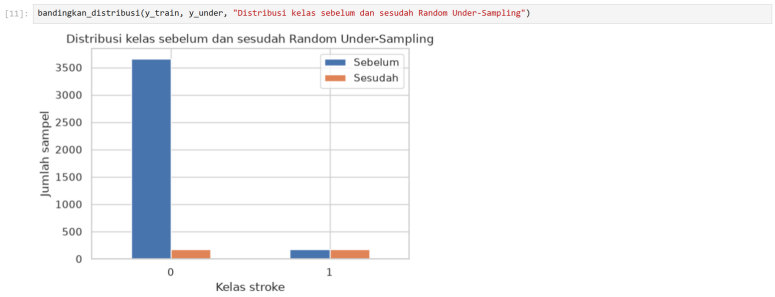
```
under = RandomUnderSampler(random_state=0)
X_under, y_under = under.fit_resample(X_train, y_train)
print("Sebelum:", pd.Series(y_train).value_counts().to_dict())
print("Sesudah:", pd.Series(y_under).value_counts().to_dict())
```

Listing 8: Penerapan `RandomUnderSampler` pada set pelatihan.

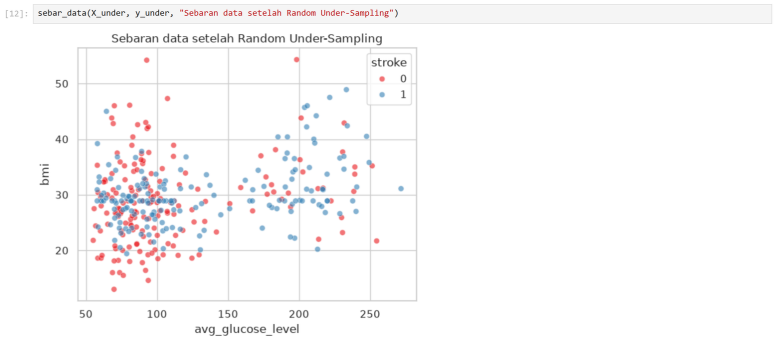


Gambar 11: Tangkapan layar distribusi kelas sebelum dan sesudah *Random Under-Sampling*.

Kelas mayoritas turun dari 3.663 menjadi 169 sampel, sehingga set pelatihan hanya menyisakan 338 baris. Sebanyak 3.494 sampel mayoritas (95,4% dari kelas tersebut) dibuang. Metode ini paling cepat, tetapi kehilangan informasi dalam jumlah besar berisiko menghapus pola yang sebenarnya penting.



Gambar 12: Perbandingan jumlah sampel per kelas sebelum dan sesudah *Random Under-Sampling*.



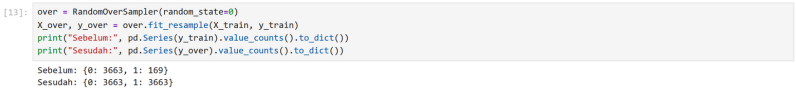
Gambar 13: Sebaran data pada bidang `avg_glucose_level` dan `bmi` setelah *Random Under-Sampling*. Titik kelas minoritas tidak lagi tenggelam di antara kelas mayoritas.

4.1.7 Random Over-Sampling

Random Over-Sampling menempuh arah sebaliknya: sampel kelas minoritas diduplikasi secara acak sampai jumlahnya menyamai kelas mayoritas.

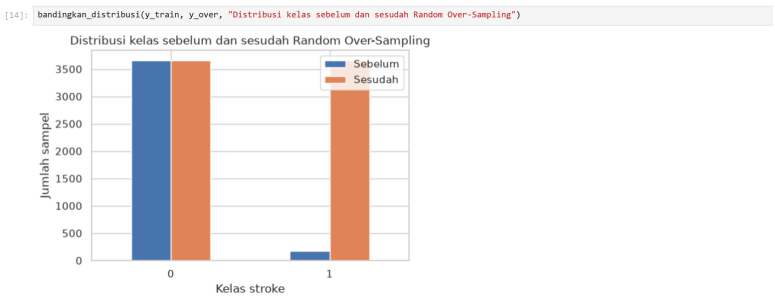
```
over = RandomOverSampler(random_state=0)
X_over, y_over = over.fit_resample(X_train, y_train)
print("Sebelum:", pd.Series(y_train).value_counts().to_dict())
print("Sesudah:", pd.Series(y_over).value_counts().to_dict())
```

Listing 9: Penerapan `RandomOverSampler` pada set pelatihan.

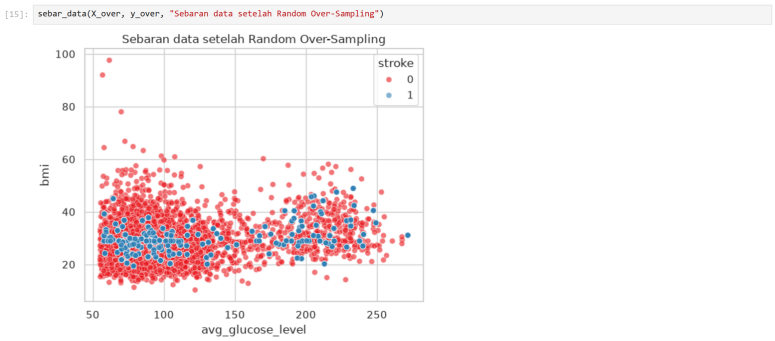


Gambar 14: Tangkapan layar distribusi kelas sebelum dan sesudah *Random Over-Sampling*.

Kelas minoritas naik dari 169 menjadi 3.663 sampel, sehingga total set pelatihan menjadi 7.326 baris. Setiap sampel minoritas rata-rata terduplikasi 21,7 kali. Cara ini mempertahankan seluruh informasi asli, tetapi duplikasi berulang membuat model cenderung menghafal sampel yang sama dan memperbesar risiko *overfitting*.



Gambar 15: Perbandingan jumlah sampel per kelas sebelum dan sesudah *Random Over-Sampling*.



Gambar 16: Sebaran data setelah *Random Over-Sampling*. Titik minoritas menumpuk tepat di posisi aslinya karena hasil duplikasi.

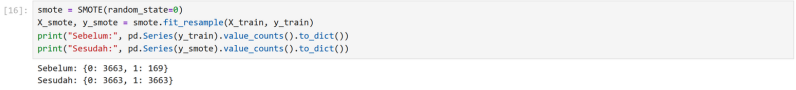
Pada Gambar 16 terlihat titik kelas minoritas menumpuk persis di koordinat yang sama. Penumpukan ini yang membedakan duplikasi dari pembangkitan data sintetis: sebaran tidak bertambah luas, hanya kepadatannya yang berlipat.

4.1.8 SMOTE

SMOTE membangkitkan sampel sintetis di antara tetangga terdekat kelas minoritas, bukan menduplikasi sampel yang ada.

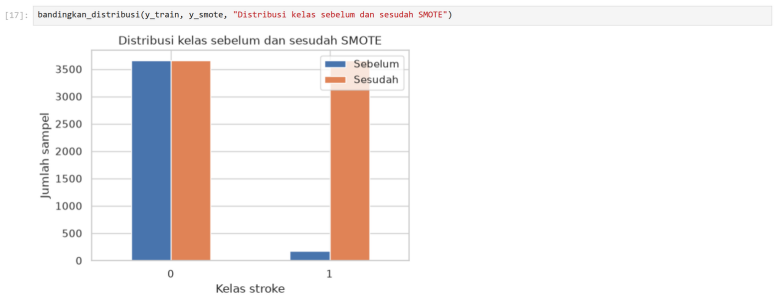
```
smote = SMOTE(random_state=0)
X_smote, y_smote = smote.fit_resample(X_train, y_train)
print("Sebelum:", pd.Series(y_train).value_counts().to_dict())
print("Sesudah:", pd.Series(y_smote).value_counts().to_dict())
```

Listing 10: Penerapan SMOTE pada set pelatihan.

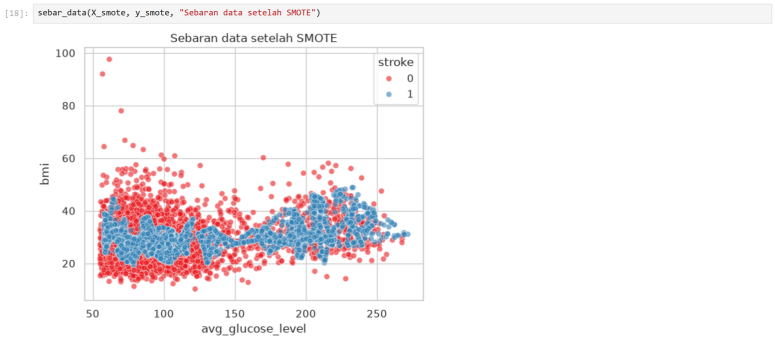


Gambar 17: Tangkapan layar distribusi kelas sebelum dan sesudah SMOTE.

Hasilnya setara dengan *Random Over-Sampling* dari sisi jumlah, yaitu 3.663 sampel per kelas dan 7.326 baris total. Bedanya ada pada asal sampel: 3.494 sampel minoritas pada SMOTE adalah data sintetis baru, bukan salinan data lama.



Gambar 18: Perbandingan jumlah sampel per kelas sebelum dan sesudah SMOTE.



Gambar 19: Sebaran data setelah SMOTE. Titik minoritas menyebar mengikuti garis penghubung antar-tetangga, tidak menumpuk di satu koordinat.

Perbedaan visualnya jelas dibandingkan Gambar 16. Pada SMOTE, wilayah kelas minoritas melebar karena titik sintetis mengisi ruang di antara sampel asli. Sebaran seperti ini memberi model variasi pola yang lebih kaya dibanding duplikasi murni.

4.1.9 ADASYN

ADASYN memberi porsi pembangkitan lebih besar pada sampel minoritas yang berdekatan dengan kelas mayoritas, karena sampel seperti itu paling sulit dipisahkan.

```
adasyn = ADASYN(random_state=0)
X_adasyn, y_adasyn = adasyn.fit_resample(X_train, y_train)
print("Sebelum:", pd.Series(y_train).value_counts().to_dict())
print("Sesudah:", pd.Series(y_adasyn).value_counts().to_dict())
```

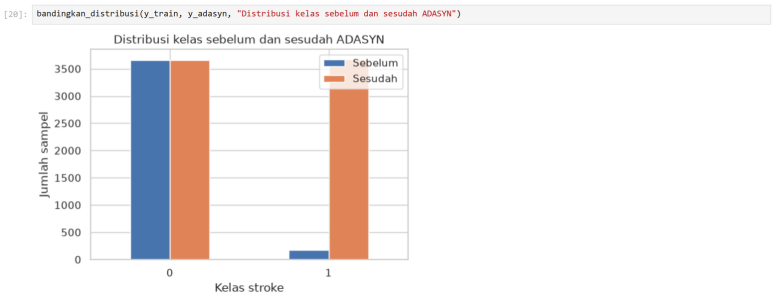
Listing 11: Penerapan ADASYN pada set pelatihan.



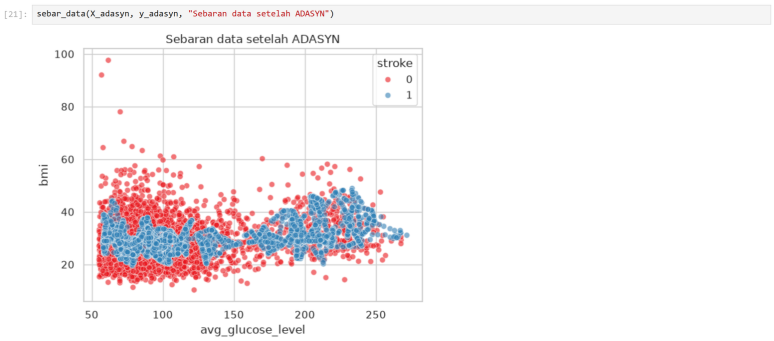
Gambar 20: Tangkapan layar distribusi kelas sebelum dan sesudah ADASYN.

Kelas minoritas berakhir dengan 3.676 sampel, tiga belas lebih banyak dari kelas mayoritas. Angka itu muncul karena mekanisme ADASYN membagi kuota sintetis menurut

bobot kesulitan tiap sampel, sehingga totalnya bisa sedikit melewati jumlah kelas mayoritas. Total set pelatihan menjadi 7.339 baris.



Gambar 21: Perbandingan jumlah sampel per kelas sebelum dan sesudah ADASYN.



Gambar 22: Sebaran data setelah ADASYN. Titik sintetis lebih terkonsentrasi di sekitar batas kedua kelas.

Tabel 3 merangkum keempat percobaan penyeimbangan pada set pelatihan yang sama.

Tabel 3: Rekapitulasi hasil empat teknik penyeimbangan kelas pada set pelatihan dataset stroke.

Teknik	**Kelas 0**	**Kelas 1**	**Total**	**Sampel baru**	**Catatan**
Tanpa balancing	3.663	169	3.832	0	Data asli
Under-sampling	169	169	338	0	3.494 sampel mayoritas dibuang
Over-sampling	3.663	3.663	7.326	3.494	Duplikasi sampel minoritas
SMOTE	3.663	3.663	7.326	3.494	Sampel sintetis di antara tetangga
ADASYN	3.663	3.676	7.339	3.507	Sintetis terpusat di batas kelas

4.1.10 Seleksi Fitur dengan Filter ANOVA

Percobaan seleksi fitur memakai *Heart Attack Analysis and Prediction Dataset* yang berisi 303 baris dan 14 kolom: 13 fitur prediktor dan satu label `output`. Dataset ini dipilih karena jumlah fiturnya cukup banyak untuk diuji, sementara labelnya masih cukup seimbang sehingga pengaruh seleksi fitur tidak tercampur dengan pengaruh penyeimbangan kelas.

```
URL_HERT = ("https://raw.githubusercontent.com/ganjar87/"
            "data_science_practice/main/heart.csv")
jantung = pd.read_csv(URL_HEART)
print("Dimensi dataset:", jantung.shape)
print(jantung["output"].value_counts())
jantung.head()
```

Listing 12: Membaca dataset heart.csv.

```
[22]: URL_HEART = ("https://raw.githubusercontent.com/ganjar87/"
                  "data_science_practice/main/heart.csv")
      jantung = pd.read_csv(URL_HEART)
      print("Dimensi dataset:", jantung.shape)
      print(jantung["output"].value_counts())
      jantung.head()

      Dimensi dataset: (303, 14)
      output
      1    165
      0    138
      Name: count, dtype: int64
```

[22]:

	age	sex	cp	trtbps	chol	fbs	restecg	thalachh	exng	oldpeak	slp	caa	thall	output
0	63	1	3	145	233	1	0	150	0	2.3	0	0	1	1
1	37	1	2	130	250	0	1	187	0	3.5	0	0	2	1
2	41	0	1	130	204	0	0	172	0	1.4	2	0	2	1
3	56	1	1	120	236	0	1	178	0	0.8	2	0	2	1
4	57	0	0	120	354	0	1	163	1	0.6	2	0	2	1

Gambar 23: Tangkapan layar pembacaan dataset heart.csv dan lima baris pertamanya.

Label `output` terbagi menjadi 165 sampel kelas 1 dan 138 sampel kelas 0. Komposisi ini masih tergolong wajar untuk klasifikasi biner.

Uji ANOVA dijalankan dengan fungsi `f_classif` yang menghitung nilai F dan p untuk setiap fitur terhadap label.

```
X_jantung = jantung.drop(columns="output")
y_jantung = jantung["output"]

skor_f, nilai_p = f_classif(X_jantung, y_jantung)
anova = (pd.DataFrame({"F": skor_f, "p_value": nilai_p}, index=X_jantung.columns)
         .sort_values("F", ascending=False).round(4))
anova
```

Listing 13: Menghitung skor F dan nilai p setiap fitur dengan uji ANOVA.

```
[23]: X_jantung = jantung.drop(columns="output")
      y_jantung = jantung["output"]

      skor_f, nilai_p = f_classif(X_jantung, y_jantung)
      anova = (pd.DataFrame({"F": skor_f, "p_value": nilai_p}, index=X_jantung.columns)
               .sort_values("F", ascending=False).round(4))
      anova
```

[23]:

	F	p_value
exng	70.9524	0.0000
cp	69.7723	0.0000
oldpeak	68.5514	0.0000
thalachh	65.1201	0.0000
caa	54.5598	0.0000
slp	40.9021	0.0000
thall	40.4077	0.0000
sex	25.7922	0.0000
age	16.1167	0.0001
trtbps	6.4582	0.0115
restecg	5.7772	0.0168
chol	2.2030	0.1388
fbs	0.2369	0.6268

Gambar 24: Tangkapan layar peringkat fitur menurut skor F hasil uji ANOVA.

Hasil lengkapnya disusun pada Tabel 4.

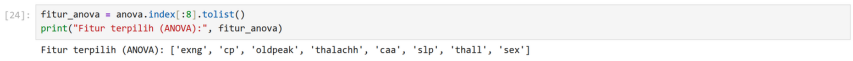
Tabel 4: Peringkat fitur dataset heart berdasarkan skor F uji ANOVA.

Fitur	F	p	**Fitur**	F	p
`exng`	70,9524	0,0000	`sex`	25,7922	0,0000
`cp`	69,7723	0,0000	`age`	16,1167	0,0001
`oldpeak`	68,5514	0,0000	`trtbps`	6,4582	0,0115
`thalachh`	65,1201	0,0000	`restecg`	5,7772	0,0168
`caa`	54,5598	0,0000	`chol`	2,2030	0,1388
`slp`	40,9021	0,0000	`fbs`	0,2369	0,6268
`thall`	40,4077	0,0000			

Delapan fitur teratas dipilih sebagai subset hasil *filter method*. Batas delapan diambil untuk menyisakan sekitar separuh fitur, sehingga pengurangan dimensi cukup terasa tanpa membuang fitur yang masih informatif.

```
fitur_anova = anova.index[:8].tolist()
print("Fitur terpilih (ANOVA):", fitur_anova)
```

Listing 14: Mengambil delapan fitur dengan skor F tertinggi.



Gambar 25: Tangkapan layar delapan fitur terpilih hasil uji ANOVA.

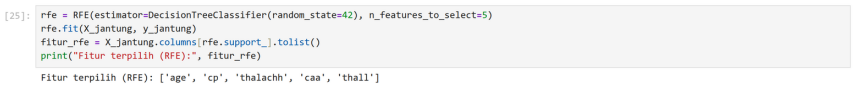
Tiga fitur terbawah, yaitu `restecg`, `chol`, dan `fbs`, memiliki $p > 0{,}01$ sehingga perbedaan rata-ratanya antar-kelas tidak cukup meyakinkan. Ketiganya otomatis tersisih oleh peringkat F dan tidak masuk subset.

4.1.11 Seleksi Fitur dengan Wrapper RFE

Recursive Feature Elimination memakai pohon keputusan sebagai penilai, membuang fitur dengan bobot terkecil pada setiap iterasi, lalu berhenti saat jumlah fitur mencapai target lima.

```
rfe = RFE(estimator=DecisionTreeClassifier(random_state=42), n_features_to_select=5)
rfe.fit(X_jantung, y_jantung)
fitur_rfe = X_jantung.columns[rfe.support_].tolist()
print("Fitur terpilih (RFE):", fitur_rfe)
```

Listing 15: Seleksi lima fitur dengan RFE.



Gambar 26: Tangkapan layar lima fitur terpilih hasil RFE.

Lima fitur yang bertahan adalah `age`, `cp`, `thalachh`, `caa`, dan `thall`. Susunan ini berbeda dari contoh pada materi kuliah yang menyisakan `sex`, `exng`, `slp`, `caa`, dan `thall`. Perbedaan itu wajar karena urutan eliminasi bergantung pada pohon yang terbentuk di setiap iterasi, dan pohon itu dipengaruhi *random_state* serta versi pustaka. Yang penting, kedua hasil sama-sama menyisakan lima fitur dengan `caa` dan `thall` sebagai irisan.

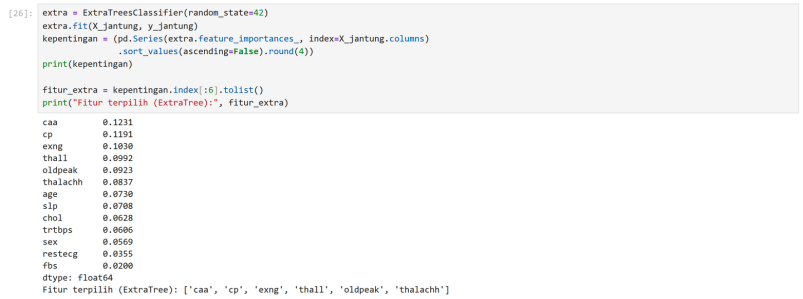
4.1.12 Seleksi Fitur dengan Embedded ExtraTree

ExtraTreesClassifier menghitung *feature importance* selama pelatihan, sehingga seleksi berjalan tanpa proses berulang yang terpisah.

```
extra = ExtraTreesClassifier(random_state=42)
extra.fit(X_jantung, y_jantung)
kepentingan = (pd.Series(extra.feature_importances_, index=X_jantung.columns)
               .sort_values(ascending=False).round(4))
print(kepentingan)

fitur_extra = kepentingan.index[:6].tolist()
print("Fitur terpilih (ExtraTree):", fitur_extra)
```

Listing 16: Menghitung *feature importance* dengan ExtraTree.



Gambar 27: Tangkapan layar peringkat *feature importance* dan enam fitur terpilih hasil ExtraTree.

Enam fitur teratas adalah `caa` (0,1231), `cp` (0,1191), `exng` (0,1030), `thall` (0,0992), `oldpeak` (0,0923), dan `thalachh` (0,0837). Himpunan ini identik dengan delapan fitur hasil ANOVA tanpa `slp` dan `sex`. Kesamaan tersebut masuk akal karena keduanya menilai kemampuan fitur memisahkan kelas, hanya metriknya yang berbeda.

4.1.13 Komparasi Metode Seleksi Fitur dan Pemeriksaan Overfitting

Ketiga subset fitur diuji dengan pengklasifikasi yang sama pada pembagian data yang sama, yaitu `DecisionTreeClassifier` dengan `random_state=42` dan rasio 80:20.

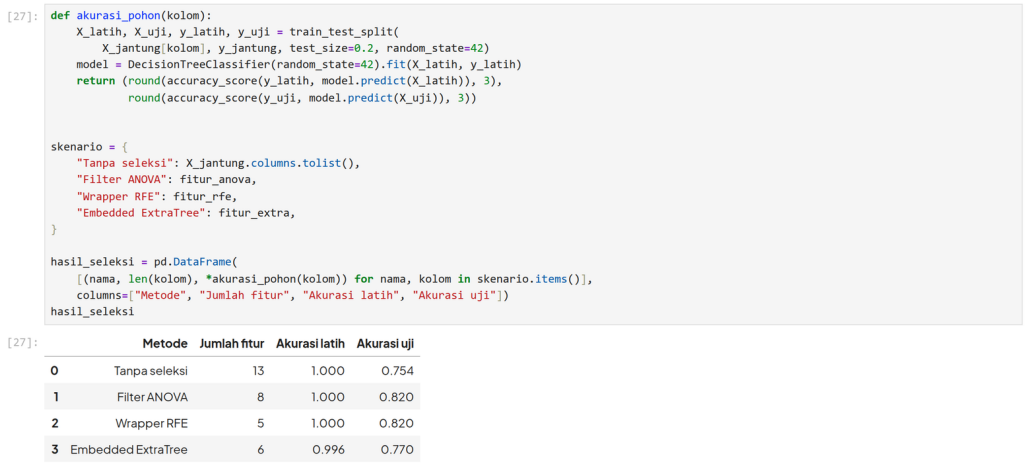
```
def akurasi_pohon(kolom):
    X_latih, X_uji, y_latih, y_uji = train_test_split(
        X_jantung[kolom], y_jantung, test_size=0.2, random_state=42)
    model = DecisionTreeClassifier(random_state=42).fit(X_latih, y_latih)
    return (round(accuracy_score(y_latih, model.predict(X_latih)), 3),
```

```
            round(accuracy_score(y_uji, model.predict(X_uji)), 3))


skenario = {
    "Tanpa seleksi": X_jantung.columns.tolist(),
    "Filter ANOVA": fitur_anova,
    "Wrapper RFE": fitur_rfe,
    "Embedded ExtraTree": fitur_extra,
}

hasil_seleksi = pd.DataFrame(
    [(nama, len(kolom), *akurasi_pohon(kolom)) for nama, kolom in skenario.items()],
    columns=["Metode", "Jumlah fitur", "Akurasi latih", "Akurasi uji"])
hasil_seleksi
```

Listing 17: Membandingkan akurasi latih dan uji pada empat skenario fitur.



[27]:

```
def akurasi_pohon(kolom):
    X_latih, X_uji, y_latih, y_uji = train_test_split(
        X_jantung[kolom], y_jantung, test_size=0.2, random_state=42)
    model = DecisionTreeClassifier(random_state=42).fit(X_latih, y_latih)
    return (round(accuracy_score(y_latih, model.predict(X_latih)), 3),
            round(accuracy_score(y_uji, model.predict(X_uji)), 3))


skenario = {
    "Tanpa seleksi": X_jantung.columns.tolist(),
    "Filter ANOVA": fitur_anova,
    "Wrapper RFE": fitur_rfe,
    "Embedded ExtraTree": fitur_extra,
}

hasil_seleksi = pd.DataFrame(
    [(nama, len(kolom), *akurasi_pohon(kolom)) for nama, kolom in skenario.items()],
    columns=["Metode", "Jumlah fitur", "Akurasi latih", "Akurasi uji"])
hasil_seleksi
```

[27]:

	Metode	Jumlah fitur	Akurasi latih	Akurasi uji
0	Tanpa seleksi	13	1.000	0.754
1	Filter ANOVA	8	1.000	0.820
2	Wrapper RFE	5	1.000	0.820
3	Embedded ExtraTree	6	0.996	0.770

Gambar 28: Tangkapan layar tabel perbandingan akurasi empat skenario seleksi fitur.

Hasilnya dirangkum pada Tabel 5.

Tabel 5: Perbandingan akurasi *Decision Tree* pada empat skenario seleksi fitur dataset heart.

Skenario	**Jumlah fitur**	**Akurasi uji**	**Fitur yang dipakai**
Tanpa seleksi	13	0,754	Seluruh fitur
Filter ANOVA	8	0,820	`exng`, `cp`, `oldpeak`, `thalachh`, `caa`, `slp`, `thall`, `sex`
Wrapper RFE	5	0,820	`age`, `cp`, `thalachh`, `caa`, `thall`
Embedded ExtraTree	6	0,770	`caa`, `cp`, `exng`, `thall`, `oldpeak`, `thalachh`

Pemangkasan fitur menaikkan akurasi uji dari 0,754 menjadi 0,820, atau 6,6 poin persen. RFE memberi hasil terbaik dari sisi efisiensi karena hanya memakai lima fitur untuk akurasi yang sama dengan ANOVA yang memakai delapan fitur. ExtraTree sedikit tertinggal pada percobaan ini meski jumlah fiturnya hanya enam.

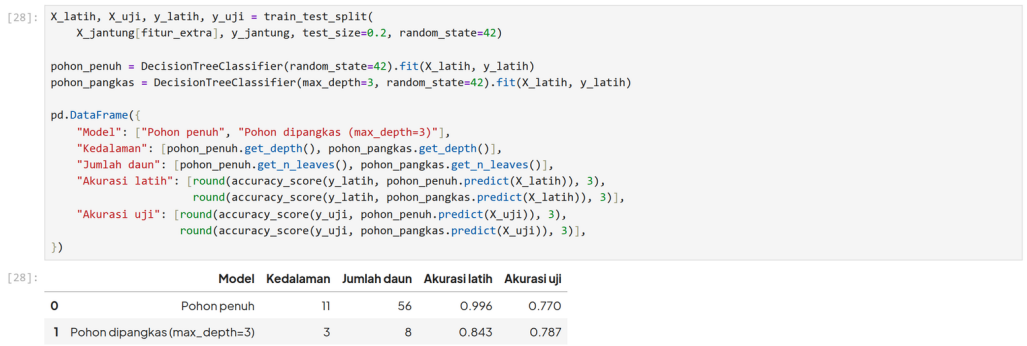
Satu hal yang mencolok pada tabel tersebut: akurasi latih menyentuh 1,000 pada tiga skenario. Pohon keputusan tanpa batas kedalaman memang sanggup menghafal seluruh data latih. Untuk memastikan jarak itu benar-benar gejala *overfitting*, saya membandingkan pohon penuh dengan pohon yang kedalamannya dibatasi.

```
X_latih, X_uji, y_latih, y_uji = train_test_split(
    X_jantung[fitur_extra], y_jantung, test_size=0.2, random_state=42)

pohon_penuh = DecisionTreeClassifier(random_state=42).fit(X_latih, y_latih)
pohon_pangkas = DecisionTreeClassifier(max_depth=3, random_state=42).fit(X_latih, y_latih)

pd.DataFrame({
    "Model": ["Pohon penuh", "Pohon dipangkas (max_depth=3)"],
    "Kedalaman": [pohon_penuh.get_depth(), pohon_pangkas.get_depth()],
    "Jumlah daun": [pohon_penuh.get_n_leaves(), pohon_pangkas.get_n_leaves()],
    "Akurasi latih": [round(accuracy_score(y_latih, pohon_penuh.predict(X_latih)), 3),
                      round(accuracy_score(y_latih, pohon_pangkas.predict(X_latih)), 3)],
    "Akurasi uji": [round(accuracy_score(y_uji, pohon_penuh.predict(X_uji)), 3),
                    round(accuracy_score(y_uji, pohon_pangkas.predict(X_uji)), 3)],
})
```

Listing 18: Membandingkan pohon penuh dengan pohon yang dipangkas.



[28]:
```
X_latih, X_uji, y_latih, y_uji = train_test_split(
    X_jantung[fitur_extra], y_jantung, test_size=0.2, random_state=42)

pohon_penuh = DecisionTreeClassifier(random_state=42).fit(X_latih, y_latih)
pohon_pangkas = DecisionTreeClassifier(max_depth=3, random_state=42).fit(X_latih, y_latih)

pd.DataFrame({
    "Model": ["Pohon penuh", "Pohon dipangkas (max_depth=3)"],
    "Kedalaman": [pohon_penuh.get_depth(), pohon_pangkas.get_depth()],
    "Jumlah daun": [pohon_penuh.get_n_leaves(), pohon_pangkas.get_n_leaves()],
    "Akurasi latih": [round(accuracy_score(y_latih, pohon_penuh.predict(X_latih)), 3),
                      round(accuracy_score(y_latih, pohon_pangkas.predict(X_latih)), 3)],
    "Akurasi uji": [round(accuracy_score(y_uji, pohon_penuh.predict(X_uji)), 3),
                    round(accuracy_score(y_uji, pohon_pangkas.predict(X_uji)), 3)],
})
```

[28]:

	Model	Kedalaman	Jumlah daun	Akurasi latih	Akurasi uji
0	Pohon penuh	11	56	0.996	0.770
1	Pohon dipangkas (max_depth=3)	3	8	0.843	0.787

Gambar 29: Tangkapan layar perbandingan kedalaman, jumlah daun, dan akurasi pohon penuh dengan pohon yang dipangkas.

Tabel 6: Pengaruh pemangkasan kedalaman terhadap selisih akurasi latih dan uji.

Model	**Kedalaman**	**Jumlah daun**	**Akurasi latih**	**Akurasi uji**
Pohon penuh	11	56	0,996	0,770
Pohon dipangkas (`max_depth=3`)	3	8	0,843	0,787

Pohon penuh tumbuh sampai kedalaman 11 dengan 56 daun. Akurasi latihnya 0,996, sementara akurasi ujinya hanya 0,770. Jarak 0,226 di antara keduanya adalah tanda pohon menghafal data latih, termasuk derau yang tidak akan muncul lagi di data baru. Setelah kedalaman dibatasi menjadi 3, jumlah daun turun ke 8, akurasi latih turun ke 0,843, tetapi akurasi uji justru naik ke 0,787. Jaraknya menyempit dari 0,226 menjadi 0,056. Model yang lebih sederhana ternyata lebih mampu menggeneralisasi.

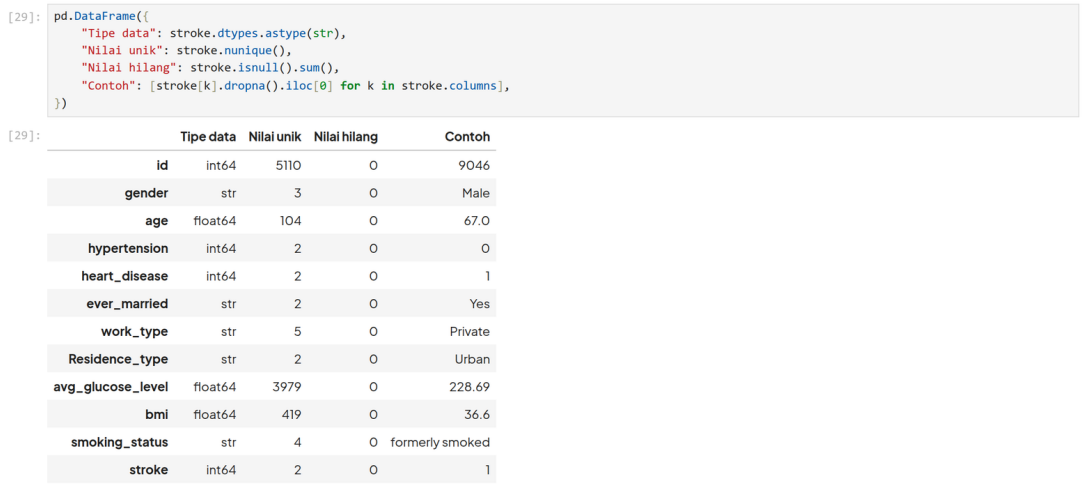
4.2 Tugas dan Analisis Praktikum

4.2.1 Tugas 1: Eksplorasi Dataset Stroke

Tugas pertama meminta penjelasan tujuan dataset dan arti setiap fitur. *Stroke Prediction Dataset* dipakai untuk membangun model klasifikasi yang memperkirakan apakah seorang pasien berpotensi mengalami stroke berdasarkan data klinis dan demografis. Dalam konteks penambangan data, dataset ini menjadi contoh kasus klasifikasi medis dengan kelas tidak seimbang yang ekstrem, yaitu 4,87% positif. Pola seperti ini menuntut perlakuan khusus pada tahap persiapan data, dan sekaligus menjadi alasan mengapa akurasi bukan metrik tunggal yang layak dipakai.

```
pd.DataFrame({
    "Tipe data": stroke.dtypes.astype(str),
    "Nilai unik": stroke.nunique(),
    "Nilai hilang": stroke.isnull().sum(),
    "Contoh": [stroke[k].dropna().iloc[0] for k in stroke.columns],
})
```

Listing 19: Merangkum tipe data, jumlah nilai unik, nilai kosong, dan contoh nilai tiap fitur.

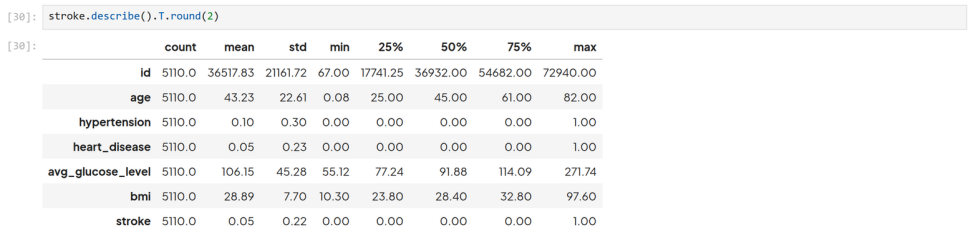


[29]:
```
pd.DataFrame({
    "Tipe data": stroke.dtypes.astype(str),
    "Nilai unik": stroke.nunique(),
    "Nilai hilang": stroke.isnull().sum(),
    "Contoh": [stroke[k].dropna().iloc[0] for k in stroke.columns],
})
```

[29]:

	Tipe data	Nilai unik	Nilai hilang	Contoh
id	int64	5110	0	9046
gender	str	3	0	Male
age	float64	104	0	67.0
hypertension	int64	2	0	0
heart_disease	int64	2	0	1
ever_married	str	2	0	Yes
work_type	str	5	0	Private
Residence_type	str	2	0	Urban
avg_glucose_level	float64	3979	0	228.69
bmi	float64	419	0	36.6
smoking_status	str	4	0	formerly smoked
stroke	int64	2	0	1

Gambar 30: Tangkapan layar ringkasan tipe data, jumlah nilai unik, nilai kosong, dan contoh nilai setiap kolom.



[30]:
```
stroke.describe().T.round(2)
```

[30]:

	count	mean	std	min	25%	50%	75%	max
id	5110.0	36517.83	21161.72	67.00	17741.25	36932.00	54682.00	72940.00
age	5110.0	43.23	22.61	0.08	25.00	45.00	61.00	82.00
hypertension	5110.0	0.10	0.30	0.00	0.00	0.00	0.00	1.00
heart_disease	5110.0	0.05	0.23	0.00	0.00	0.00	0.00	1.00
avg_glucose_level	5110.0	106.15	45.28	55.12	77.24	91.88	114.09	271.74
bmi	5110.0	28.89	7.70	10.30	23.80	28.40	32.80	97.60
stroke	5110.0	0.05	0.22	0.00	0.00	0.00	0.00	1.00

Gambar 31: Tangkapan layar statistik deskriptif fitur numerik dataset stroke.

Penjelasan setiap fitur disusun pada Tabel 7.

Tabel 7: Arti setiap fitur pada *Stroke Prediction Dataset*.

Fitur	**Tipe**	**Arti dan rentang nilai**
`id`	Bilangan bulat	Nomor unik pasien, 5.110 nilai berbeda. Tidak dipakai sebagai fitur karena tidak punya makna klinis.
`gender`	Kategorikal	Jenis kelamin pasien: *Male*, *Female*, *Other*.
`age`	Pecahan	Usia pasien dalam tahun, rentang 0,08 sampai 82 dengan rata-rata 43,23.
`hypertension`	Biner	Riwayat hipertensi: 0 tidak, 1 ya. Rata-rata 0,10 berarti sekitar 10% pasien hipertensi.
`heart_disease`	Biner	Riwayat penyakit jantung: 0 tidak, 1 ya. Rata-rata 0,05.
`ever_married`	Kategorikal	Status pernah menikah: *Yes* atau *No*.
`work_type`	Kategorikal	Jenis pekerjaan: *Private*, *Self-employed*, *Govt_job*, *children*, *Never_worked*.
`Residence_type`	Kategorikal	Tipe tempat tinggal: *Urban* atau *Rural*.
`avg_glucose_level`	Pecahan	Rata-rata kadar glukosa darah dalam mg/dL, rentang 55,12 sampai 271,74 dengan rata-rata 106,15.
`bmi`	Pecahan	Indeks massa tubuh dalam kg/m^2, rentang 10,30 sampai 97,60 dengan rata-rata 28,89. Memiliki 201 nilai kosong.
`smoking_status`	Kategorikal	Status merokok: *formerly smoked*, *never smoked*, *smokes*, *Unknown*.
`stroke`	Biner	Label target: 1 pasien mengalami stroke, 0 tidak. Hanya 4,87% bernilai 1.

Dua fitur perlu perhatian khusus. `bmi` memiliki 201 nilai kosong sehingga wajib diimputasi sebelum dipakai. `age` memiliki nilai minimum 0,08 tahun, yang secara medis tidak lazim untuk data pasien stroke dan bisa jadi hasil pencatatan dalam satuan berbeda. Saya membiarkan nilai tersebut apa adanya karena modul tidak menginstruksikan pembersihan lebih lanjut, tetapi ketidakwajaran ini perlu dicatat sebagai keterbatasan dataset.

4.2.2 Tugas 2: Komparasi Balancing pada Dataset Kedua

Tugas kedua meminta perbandingan performa model sebelum dan sesudah penyeimbangan pada dataset lain. Saya memilih *Wine Quality (red)* dari UCI karena labelnya bisa dibentuk menjadi klasifikasi biner yang timpang tanpa rekayasa berlebihan.

Label asli dataset ini berupa skor kualitas ordinal 3 sampai 8 dengan sebaran yang tidak merata: skor 5 dan 6 mendominasi, sedangkan skor 7 hanya 199 baris dan skor 8 hanya 18 baris. Saya mengubah label tersebut menjadi dua kelas, yaitu kualitas premium untuk skor 7

ke atas dan kualitas standar untuk skor di bawah 7. Hasilnya kelas minoritas berisi 217 baris (13,57%) dan kelas mayoritas 1.382 baris (86,43%), dengan rasio 6,37 banding 1.

```
URL_WINE = "https://raw.githubusercontent.com/plotly/datasets/master/winequality-red.csv"
anggur = pd.read_csv(URL_WINE)
anggur["premium"] = (anggur["quality"] >= 7).astype(int)
anggur = anggur.drop(columns="quality")

print("Dimensi dataset:", anggur.shape)
print(anggur["premium"].value_counts())
print("Rasio ketidakseimbangan:", round(1382 / 217, 2), ": 1")
```

Listing 20: Membaca dataset Wine Quality dan membentuk label biner premium.



Gambar 32: Tangkapan layar dimensi, distribusi kelas, dan rasio ketidakseimbangan dataset Wine Quality.

Data dibagi 75:25 dengan `stratify` agar proporsi kelas pada kedua bagian tetap sama. Pemakaian `stratify` di sini berbeda dari percobaan stroke yang mengikuti modul, karena pada dataset ini jumlah kelas minoritas cukup besar sehingga stratifikasi tidak mengubah struktur data secara berarti.

```
X_anggur = anggur.drop(columns="premium")
y_anggur = anggur["premium"]
X_latih_a, X_uji_a, y_latih_a, y_uji_a = train_test_split(
    X_anggur, y_anggur, test_size=0.25, random_state=42, stratify=y_anggur)

print("Latih:", pd.Series(y_latih_a).value_counts().to_dict())
print("Uji:", pd.Series(y_uji_a).value_counts().to_dict())
```

Listing 21: Membagi data dengan stratifikasi kelas.



Gambar 33: Tangkapan layar distribusi kelas pada set pelatihan dan pengujian setelah stratifikasi.

Set pelatihan berisi 1.199 baris (1.036 kelas mayoritas dan 163 kelas minoritas), sedangkan set pengujian berisi 400 baris (346 dan 54). Dua pengklasifikasi dipakai, yaitu *Decision Tree* dan *Random Forest*, agar pola perubahannya terlihat konsisten dan tidak bergantung pada satu model.

```python
def evaluasi(model, X_latih, y_latih, X_uji, y_uji, label):
    model.fit(X_latih, y_latih)
    prediksi = model.predict(X_uji)
    return {
        "Model": label,
        "Accuracy": round(accuracy_score(y_uji, prediksi), 4),
        "Precision": round(precision_score(y_uji, prediksi), 4),
        "Recall": round(recall_score(y_uji, prediksi), 4),
        "F1-Score": round(f1_score(y_uji, prediksi), 4),
    }


baris = []
for nama_model, pembuat in [("Decision Tree", DecisionTreeClassifier),
                            ("Random Forest", RandomForestClassifier)]:
    baris.append(evaluasi(pembuat(random_state=42), X_latih_a, y_latih_a,
                          X_uji_a, y_uji_a, f"{nama_model} tanpa balancing"))
    for nama_sampler, sampler in [("SMOTE", SMOTE(random_state=42)),
                                  ("ADASYN", ADASYN(random_state=42))]:
        X_res, y_res = sampler.fit_resample(X_latih_a, y_latih_a)
        baris.append(evaluasi(pembuat(random_state=42), X_res, y_res,
                              X_uji_a, y_uji_a, f"{nama_model} + {nama_sampler}"))

tabel_performa = pd.DataFrame(baris)
tabel_performa
```

Listing 22: Menguji dua pengklasifikasi pada tiga kondisi penyeimbangan.

[33]:

```python
def evaluasi(model, X_latih, y_latih, X_uji, y_uji, label):
    model.fit(X_latih, y_latih)
    prediksi = model.predict(X_uji)
    return {
        "Model": label,
        "Accuracy": round(accuracy_score(y_uji, prediksi), 4),
        "Precision": round(precision_score(y_uji, prediksi), 4),
        "Recall": round(recall_score(y_uji, prediksi), 4),
        "F1-Score": round(f1_score(y_uji, prediksi), 4),
    }


baris = []
for nama_model, pembuat in [("Decision Tree", DecisionTreeClassifier),
                            ("Random Forest", RandomForestClassifier)]:
    baris.append(evaluasi(pembuat(random_state=42), X_latih_a, y_latih_a,
                          X_uji_a, y_uji_a, f"{nama_model} tanpa balancing"))
    for nama_sampler, sampler in [("SMOTE", SMOTE(random_state=42)),
                                  ("ADASYN", ADASYN(random_state=42))]:
        X_res, y_res = sampler.fit_resample(X_latih_a, y_latih_a)
        baris.append(evaluasi(pembuat(random_state=42), X_res, y_res,
                              X_uji_a, y_uji_a, f"{nama_model} + {nama_sampler}"))

tabel_performa = pd.DataFrame(baris)
tabel_performa
```

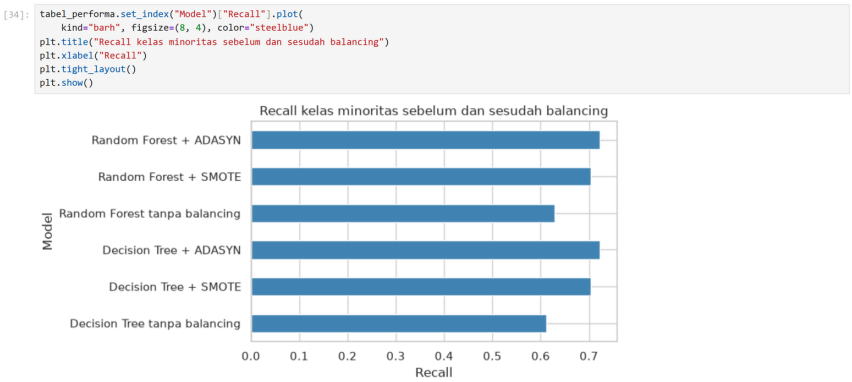
[33]:

	Model	Accuracy	Precision	Recall	F1-Score
0	Decision Tree tanpa balancing	0.8950	0.6111	0.6111	0.6111
1	Decision Tree + SMOTE	0.8775	0.5352	0.7037	0.6080
2	Decision Tree + ADASYN	0.8700	0.5132	0.7222	0.6000
3	Random Forest tanpa balancing	0.9400	0.8947	0.6296	0.7391
4	Random Forest + SMOTE	0.9025	0.6230	0.7037	0.6609
5	Random Forest + ADASYN	0.9050	0.6290	0.7222	0.6724

Gambar 34: Tangkapan layar tabel performa dua pengklasifikasi pada tiga kondisi penyeimbangan.

Tabel 8: Performa model sebelum dan sesudah penyeimbangan pada dataset Wine Quality.

Model	Accuracy	Precision	Recall	F1-Score
Decision Tree tanpa balancing	0,8950	0,6111	0,6111	0,6111
Decision Tree + SMOTE	0,8775	0,5352	0,7037	0,6080
Decision Tree + ADASYN	0,8700	0,5132	0,7222	0,6000
Random Forest tanpa balancing	0,9400	0,8947	0,6296	0,7391
Random Forest + SMOTE	0,9025	0,6230	0,7037	0,6609
Random Forest + ADASYN	0,9050	0,6290	0,7222	0,6724



Gambar 35: Perbandingan *recall* kelas minoritas pada enam konfigurasi model.

Polanya konsisten pada kedua pengklasifikasi. Akurasi turun, *recall* naik. Pada *Decision Tree*, akurasi bergerak dari 0,8950 ke 0,8775 (SMOTE) dan 0,8700 (ADASYN), sementara *recall* naik dari 0,6111 ke 0,7037 dan 0,7222. Pada *Random Forest*, akurasi turun dari 0,9400 ke 0,9025 dan 0,9050, sedangkan *recall* naik dari 0,6296 ke 0,7037 dan 0,7222.

Penurunan akurasi adalah konsekuensi yang diharapkan dari pergeseran titik operasi ini. Model yang lebih jarang melewatkan sampel premium memang harus menerima lebih banyak tebakan salah pada kelas standar, dan itulah harga yang dibayar untuk menaikkan *recall* sekitar 9 sampai 11 poin persen. Pada konteks penyaringan kualitas, melewatkan anggur premium yang sebenarnya laku mahal biasanya lebih merugikan dibanding salah menaikkan satu produk standar ke kategori premium.

Perubahan *precision* menjelaskan mekanismenya. Nilai *precision* turun cukup tajam, dari 0,8947 menjadi 0,6230 pada *Random Forest*. Model jadi lebih agresif menyatakan sebuah sampel premium, sehingga sebagian prediksi positif ternyata keliru. F1-Score sebagai rata-rata harmonik keduanya ikut turun karena penurunan *precision* lebih besar daripada kenaikan *recall*. Penyeimbangan memindahkan titik operasi model ke arah sensitivitas, dan pemilihan teknik tetap harus mempertimbangkan biaya kesalahan di domain yang dihadapi. ADASYN sedikit lebih unggul dibanding SMOTE pada *recall* di kedua model, yaitu 0,7222 berbanding 0,7037. Selisihnya berasal dari cara ADASYN memusatkan sampel sintetis di sekitar batas keputusan. Namun keunggulan itu diikuti *precision* yang lebih rendah, sehingga F1-Score keduanya hampir setara.

Kesimpulan

Berdasarkan praktikum yang telah saya laksanakan pada Pertemuan 7, diperoleh kesimpulan sebagai berikut:

1. *Imbalanced class* membuat model bias ke kelas mayoritas. Pada *Stroke Prediction Dataset*, rasio kelas mencapai 19,52 banding 1 dengan hanya 4,87% pasien mengalami stroke, sehingga akurasi menjadi metrik yang menyesatkan dan *recall* kelas minoritas lebih layak dijadikan acuan.
2. Empat teknik penyeimbangan menghasilkan karakter data yang berbeda meski tujuannya sama. *Random Under-Sampling* memangkas set pelatihan dari 3.832 menjadi 338 baris dan membuang 95,4% sampel mayoritas. *Random Over-Sampling*, SMOTE, dan ADASYN sama-sama menaikkan jumlah sampel minoritas menjadi sekitar 3.663, tetapi hanya SMOTE dan ADASYN yang menambah variasi karena datanya sintetis.
3. ADASYN membangkitkan 3.507 sampel sintetis dan berakhir dengan 3.676 sampel minoritas, tiga belas lebih banyak dari kelas mayoritas. Kelebihan itu muncul karena kuota sintetis dibagi menurut bobot kesulitan tiap sampel, bukan dibagi rata.
4. Penyeimbangan hanya boleh diterapkan pada set pelatihan. Menerapkannya pada set pengujian akan membuat evaluasi kehilangan makna karena data uji tidak lagi mencerminkan sebaran asli.
5. Seleksi fitur menaikkan akurasi uji *Decision Tree* pada dataset heart dari 0,754 menjadi 0,820, atau 6,6 poin persen. *Wrapper* RFE mencapai hasil tertinggi dengan lima fitur saja, sehingga paling efisien di antara ketiga pendekatan.
6. *Filter* ANOVA dan *embedded* ExtraTree memilih himpunan fitur yang hampir sama, yaitu `caa`, `cp`, `exng`, `thall`, `oldpeak`, dan `thalachh`. Keduanya menilai kemampuan fitur memisahkan kelas, hanya metriknya berbeda.
7. *Overfitting* terlihat dari jarak akurasi latih dan uji. Pohon penuh mencapai 0,996 pada data latih tetapi hanya 0,770 pada data uji. Pemangkasan kedalaman menjadi 3 menurunkan akurasi latih ke 0,843 dan menaikkan akurasi uji ke 0,787, sehingga jaraknya menyempit dari 0,226 menjadi 0,056.
8. Pada dataset Wine Quality, penyeimbangan menaikkan *recall* kelas minoritas 9 sampai 11 poin persen tetapi menurunkan akurasi 2 sampai 3,5 poin persen.

Daftar Pustaka

[1] A. Istiana and M. Mustafiril, "Perbandingan Metode Klasifikasi pada Data dengan Imbalance Class dan Missing Value," *JURNAL INFORMATIKA*, pp. 101–108, 2023.

[2] Normawati and T. Prayogi, "Implementasi Naïve Bayes Classifier dan Confusion Matrix pada Analisis Sentimen Berbasis Teks pada Twitter," *J-SAKTI (Jurnal Sains Komputer & Informatika)*, pp. 697–711, 2021.

[3] R. Septiana, A. Susanto, and Tukiyat, "Analisis Sentimen Vaksinasi Covid-19 pada Twitter Menggunakan Naive Bayes Classifier dengan Feature Selection Chi-Squared Statistic dan Particle Swarm Optimization," *Jurnal Sistem Komputer dan Kecerdasan Buatan*, pp. 49–56, 2021.

[4] R. Pratama and K. Kusnawi, "Komparasi Algoritma Supervised Learning dan Feature Selection pada Klasifikasi Penyakit Gagal Jantung," *Indonesian Journal of Computer Science*, pp. 3722–3733, 2023.

[5] I. Guyon and A. Elisseeff, "An Introduction to Variable and Feature Selection," *Journal of Machine Learning Research*, vol. 3, pp. 1157–1182, 2003. [Online]. Tersedia: `https://www.jmlr.org/papers/v3/guyon03a.html`.

[6] R. Kohavi and G. H. John, "Wrappers for Feature Subset Selection," *Artificial Intelligence*, vol. 97, no. 1–2, pp. 273–324, 1997. [Online]. Tersedia: `https://doi.org/10.1016/S0004-3702(97)00043-X`.

[7] I. Guyon, J. Weston, S. Barnhill, and V. Vapnik, "Gene Selection for Cancer Classification Using Support Vector Machines," *Machine Learning*, vol. 46, no. 1, pp. 389–422, 2002. [Online]. Tersedia: `https://doi.org/10.1023/A:1012487302797`.

[8] R. Tibshirani, "Regression Shrinkage and Selection via the Lasso," *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 58, no. 1, pp. 267–288, 1996.

[9] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001. [Online]. Tersedia: `https://doi.org/10.1023/A:1010933404324`.

[10] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002. [Online]. Tersedia: `https://doi.org/10.1613/jair.953`.

[11] H. He, Y. Bai, E. A. Garcia, and S. Li, "ADASYN: Adaptive Synthetic Sampling Approach for Imbalanced Learning," in *IEEE International Joint Conference on Neural Networks (IJCNN)*, pp. 1322–1328, 2008. [Online]. Tersedia: https://web.archive.org/web/2024/https://doi.org/10.1109/IJCNN.2008.4633969.

[12] N. L. P. Wijayanti, I. W. Kencana, and I. M. Sumarjaya, "SMOTE: Potensi dan Kekurangannya pada Survei," *E-Jurnal Matematika*, pp. 235–240, 2021.

[13] A. P. Ramadhanti, H. B. Santoso, and T. Widiharih, "Perbandingan SMOTE dan ADASYN pada Data Imbalance untuk Klasifikasi Rumah Tangga Miskin Kabupaten Temanggung dengan Algoritma K-Nearest Neighbor," *JURNAL GAUSSIAN*, pp. 499–505, 2022.

[14] G. Lemaître, F. Nogueira, and C. K. Aridas, "Imbalanced-learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning," *Journal of Machine Learning Research*, vol. 18, no. 17, pp. 1–5, 2017. [Online]. Tersedia: https://jmlr.org/papers/v18/16-365.html.

[15] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Tersedia: https://jmlr.org/papers/v12/pedregosa11a.html.

[16] P. Cortez, A. Cerdeira, F. Almeida, T. Matos, and J. Reis, "Modeling Wine Preferences by Data Mining from Physicochemical Properties," *Decision Support Systems*, vol. 47, no. 4, pp. 547–553, 2009. [Online]. Tersedia: https://doi.org/10.1016/j.dss.2009.05.016.

[17] Fedesoriano, "Stroke Prediction Dataset," Kaggle. [Online]. Tersedia: https://www.kaggle.com/datasets/fedesoriano/stroke-prediction-dataset.

[18] R. R. Pritom, "Heart Attack Analysis and Prediction Dataset," Kaggle. [Online]. Tersedia: https://www.kaggle.com/datasets/rashikrahmanpritom/heart-attack-analysis-prediction-dataset.

[19] P. Cortez, "Wine Quality," UCI Machine Learning Repository. [Online]. Tersedia: https://archive.ics.uci.edu/dataset/186/wine+quality.

[20] Scikit-learn Developers, "Feature Selection," [Online]. Tersedia: https://scikit-learn.org/stable/modules/feature_selection.html.

[21] Imbalanced-learn Developers, "imbalanced-learn documentation," [Online]. Tersedia: https://imbalanced-learn.org/stable/.

[22] Python Software Foundation, "Python 3 Documentation," [Online]. Tersedia: https://docs.python.org/3/.

[23] The pandas Development Team, “pandas documentation,” [Online]. Tersedia: https://pandas.pydata.org/docs/.

[24] NumPy Developers, “NumPy user guide,” [Online]. Tersedia: https://numpy.org/doc/stable/.

[25] The Matplotlib Development Team, “Matplotlib Documentation,” [Online]. Tersedia: https://matplotlib.org/stable/contents.html.

[26] M. Waskom, “Seaborn: statistical data visualization,” [Online]. Tersedia: https://seaborn.pydata.org/.

[27] H. R. Prasetya, “Praktikum 7: Imbalanced Dataset dan Feature Selection (Notebook),” Google Colaboratory, 2026. [Online]. Tersedia: https://colab.research.google.com/github/hafidzrizqullahprasetya/PPD/blob/main/pertemuan-7/praktikum-7-imbalanced-feature-selection.ipynb.